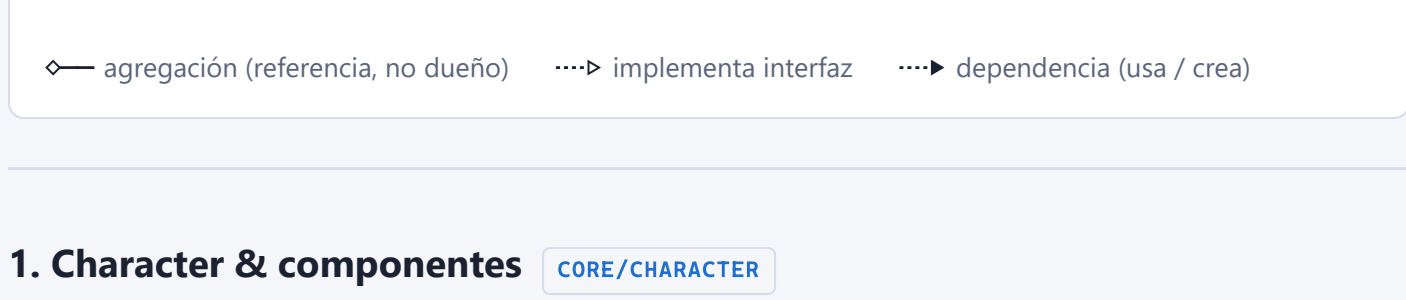


Diagrama de clases de la arquitectura principal

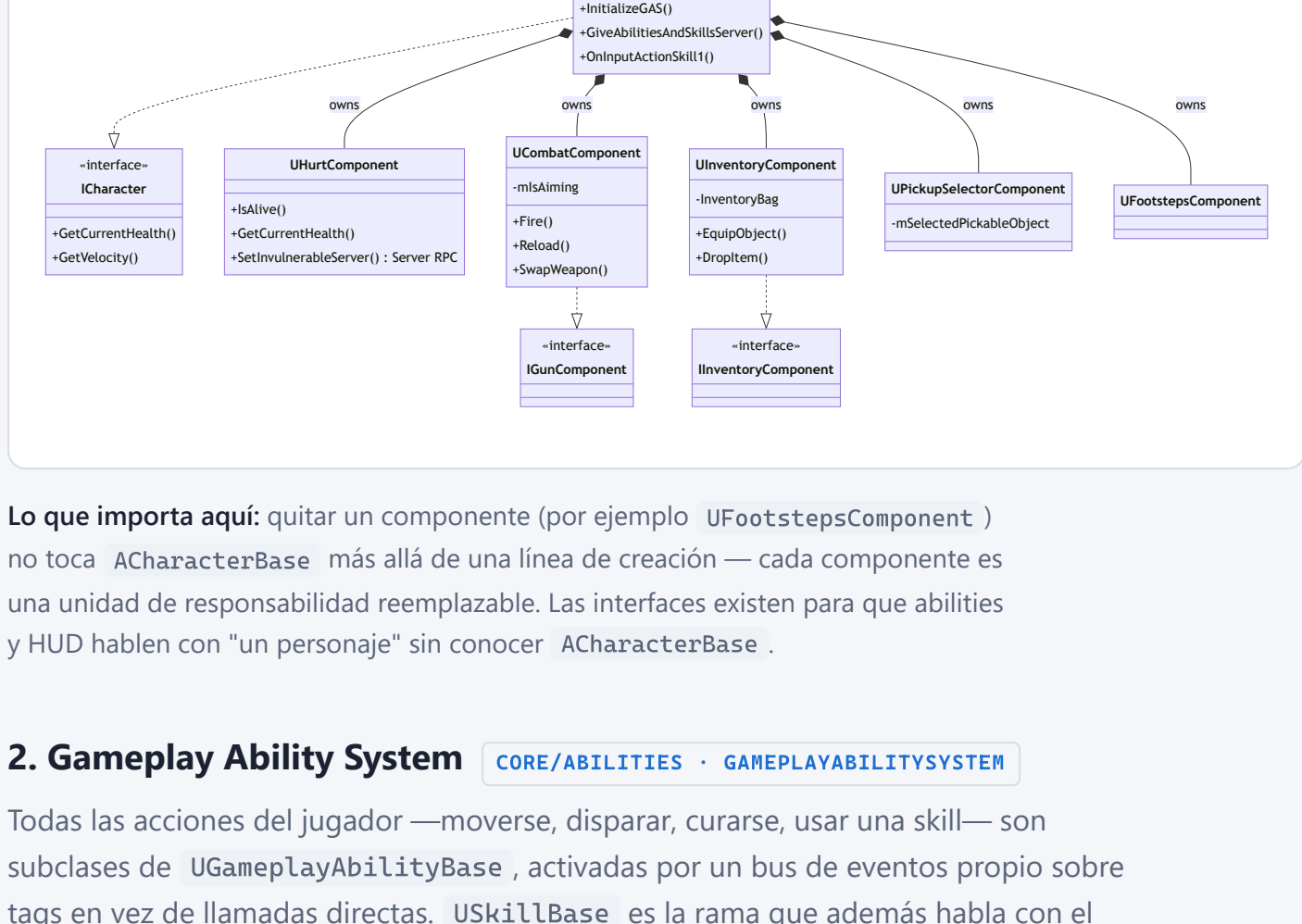
→ herencia (subclase de) ◀ composición (dueño del ciclo de vida)



interfaces (`ICharacter` , `IGunComponent` , `IInventory`)
composición, no herencia, lo que arma al personaje.

```

classDiagram
    class AFirstPersonCharacter {
        -ConvertIntoMesh(IP())
        -HideMesh()
    }
    class ACharacterBase {
        #mControlRotation
    }
    AFirstPersonCharacter --|> ACharacterBase
  
```



HUD para el cooldown.

```
classDiagram
    class UIAbilitySystemComponentBase {
        +ApplyGameplayEffectToTarget()
        +RegisterGameplayEvent()
        +SendGameplayEvent()
        +ServerHandleGameplayEvent() : Server RPC
    }
    class UIAbilitySystemComponent {
        +<vacio, sin implementar>
    }
    class USkillCooldown {
    }
    UIAbilitySystemComponentBase <|-- UIAbilitySystemComponent
    UIAbilitySystemComponentBase <|-- USkillCooldown
```

The diagram illustrates the class hierarchy for the HUD cooldown system. The base class, **UIAbilitySystemComponentBase**, defines the following methods:

- `+ApplyGameplayEffectToTarget()`
- `+RegisterGameplayEvent()`
- `+SendGameplayEvent()`
- `+ServerHandleGameplayEvent() : Server RPC`

Two subclasses inherit from **UIAbilitySystemComponentBase**:

- UIAbilitySystemComponent**: This class is marked as `<vacio, sin implementar>` (empty, not implemented).
- USkillCooldown**: This class is also marked as `<vacio, sin implementar>`.

```

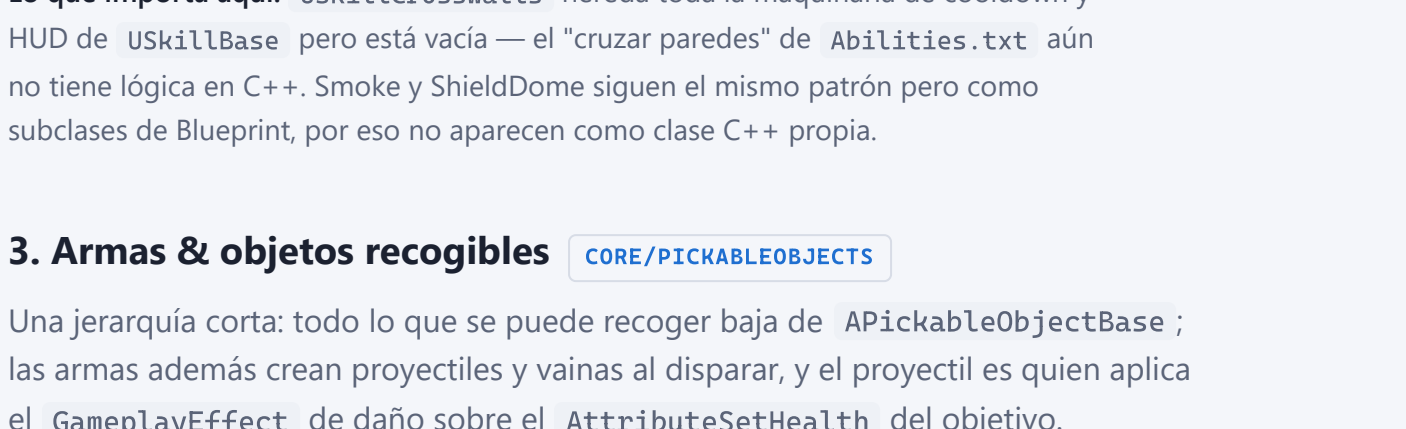
classDiagram
    class UGameplayAbilityBase
    class UAbilitySystemComponent
    class UAbilityHealth {
        +Health
    }
    class UAbilityMacSpeed {
        +MacSpeed
    }
    class UAbilityWidgetUID {
        +WidgetUID
    }
    class UAbilityOnStartSkillCooldown {
        +NotifyOnStartSkillCooldown
    }
    class UAbilityOnStartSkillCooldown

    UGameplayAbilityBase <|-- UAbilitySystemComponent
    UGameplayAbilityBase <|-- UAbilityHealth
    UGameplayAbilityBase <|-- UAbilityMacSpeed
    UGameplayAbilityBase <|-- UAbilityWidgetUID
    UGameplayAbilityBase <|-- UAbilityOnStartSkillCooldown
    UGameplayAbilityBase <|-- UAbilityOnStartSkillCooldown
  
```

```

classDiagram
    class UAttributeList {
        +UAttribute* Attributes
    }
    class UAbility {
        +AbilityType ID
        +UCharacter* Owner
        +GetCharacter()
    }
    class UGameplayAbility {
        +UEngine*
    }
    UAbility --> UGameplayAbility
    UAbility --> UAttributeList
    UGameplayAbility --> UAttributeList
  
```

The diagram illustrates the relationships between three classes: **UAttributeList**, **UAbility**, and **UGameplayAbility**. **UAbility** is a base class for **UGameplayAbility**. **UAbility** has a collection of **UAttribute** objects (indicated by a hollow diamond on the association line) and a reference to a **UCharacter** object. **UGameplayAbility** inherits from **UAbility** and has a reference to a **UEngine** object. Both **UAbility** and **UGameplayAbility** have associations with **UAttributeList**.



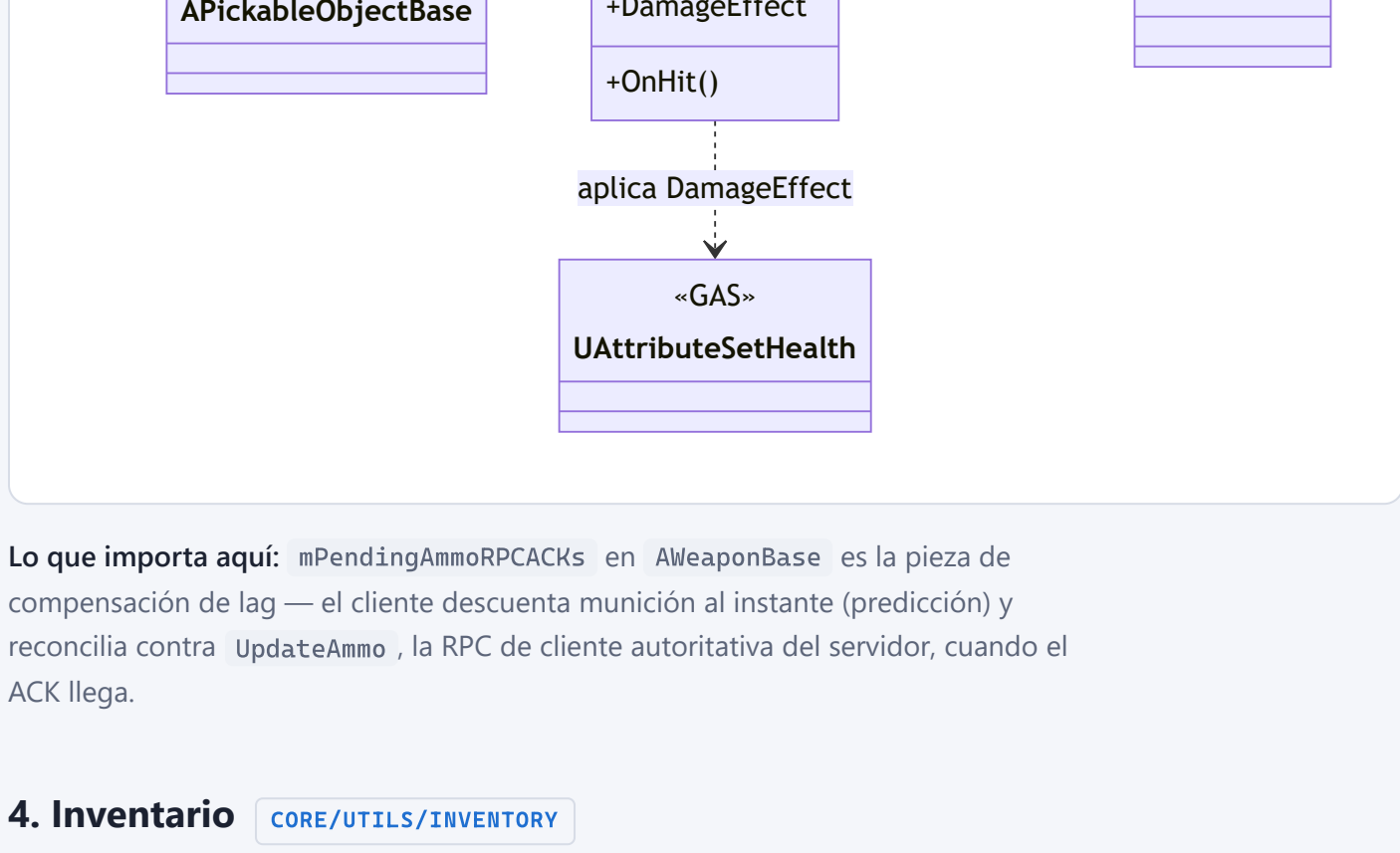
```

classDiagram
    class AWeaponBase {
        +mAmmo
        +MagazineCapacity
    }
    class AAmmo
    class AAreaProjectile
    class AWeaponBase
    AWeaponBase "0..*" -- "0..*" AAmmo
    AWeaponBase "0..*" -- "0..*" AAreaProjectile

```

- +ServerF
- +Multica

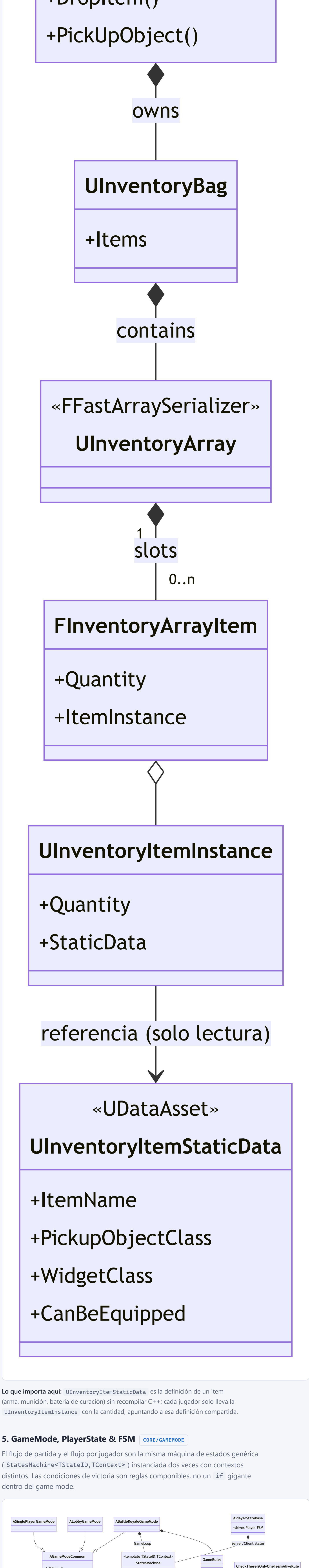
The diagram shows a Client RPC method `+UpdateAmmo()` with a dashed arrow pointing to `«IPickupObject»` and a solid arrow pointing to `AProjectileBase`. `AProjectileBase` has a dashed arrow pointing to `«IPickupObject»` labeled "spawna al dispar" and a solid arrow pointing to another `AProjectileBase` box labeled "spawna".



El inventario separa dato estático (qué es un ítem, `UDataAsset` configurab diseño) de instancia en tiempo de ejecución (cuántos tengo), y esa instancia dentro de un array replicado tipo `FFastArraySerializer` para que solo c por red los slots que realmente cambiaron.

UInvento

```
+EquipObject()
-DropItem()
```



```

classDiagram
    class BattleRoyaleContext {
        +PostLogin()
        +CanPlayerCauseDamageTo()
    }
    class PlayerStateContext {
        +Update()
        +ForceState()
    }
    class PlayerGameContext {
        +AddMusic()
        +Execute()
    }
    BattleRoyaleContext ..> PlayerStateContext : opera sobre
    PlayerGameContext --> "0..n" PlayerStateContext
  
```

Lo que importa aquí: la misma clase `StateMachine` se reutiliza en el modo de battle royale, el modo un-jugador y el estado por jugador. `TContext`, que lo invoca. Añadir una regla de victoria nueva es

implementa `IGameRule` , no un cambio en `GameRules` .

Generado a partir de `Source/BattleRoyale/core` el 2026-08-10. Clases marcadas como stub o vacías (`USkillCrossWalls`) se muestran tal cual están en el repo, no como diseño objetivo.